

Data Models

An Introduction

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

RideLink: A Ride-Hailing Platform

RideLink stores user profiles in a **relational database**.

A passenger profile includes name, phone, saved addresses, payment methods, ride preferences, and active promotions. A driver profile adds vehicle details, licence information, and service areas.

That's at least **8 normalised tables** — five for passengers (`Users`, `Addresses`, `PaymentMethods`, `Preferences`, `Promotions`) and three more for drivers (`Vehicles`, `Licenses`, `ServiceAreas`).

Loading one passenger's profile for the home screen already requires a **5-table JOIN** across all five passenger tables.

RideLink: Adding a New Feature

RideLink's schema — three of the five passenger tables, connected through `user_id`:

Users		
user_id	name	phone
P-3921	Mei Lin	+65-9123

Addresses			
addr_id	user_id	label	street
1	P-3921	Home	12 Orchard Rd
2	P-3921	Office	88 Robinson Rd

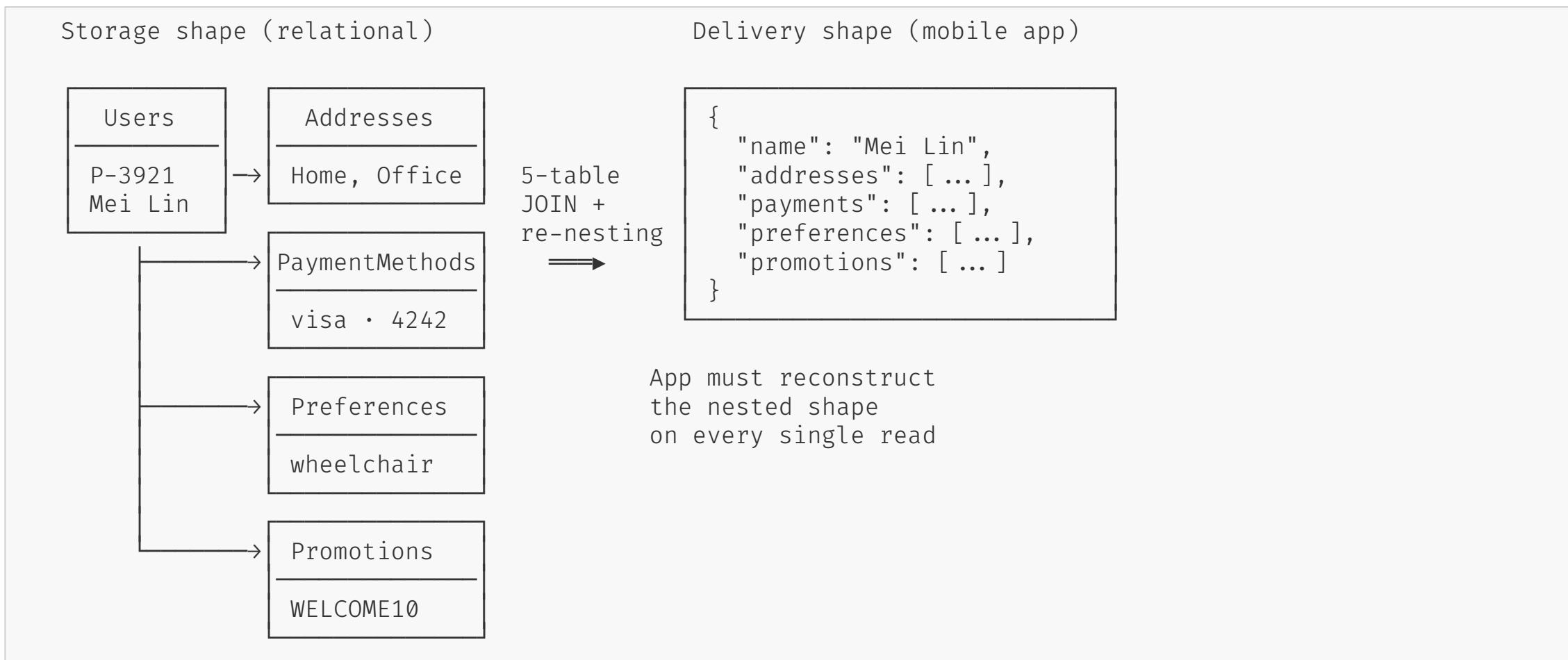
PaymentMethods				
pm_id	user_id	type	last4	is_default
1	P-3921	visa	4242	true

The product team wants to add accessibility preferences.

That means: **new table** → **new JOIN** → **new migration** → **redeployment**.

The schema is correct. The constraints are enforced. So what's wrong?

RideLink: Shape Mismatch



The storage shape does not match the delivery shape. This is a **shape mismatch** — not a bug, a **design problem**.

NeoBank: A Digital Bank

NeoBank stores its general ledger in a **document database**.

Each transaction is a self-contained JSON document:

```
{  
  "from": "A-1001",  
  "to": "A-2049",  
  "amount": 500.00,  
  "currency": "SGD"  
}
```

Each account also has a **balance document** that the application updates after every transfer. Reads are fast — one key lookup returns the full transaction or the current balance.

Then the auditor asks a question...

NeoBank: The Auditor's Question

"Show me the running balance of account A-1001 across all transactions for the month."

In a relational database, this is one query — `SUM ... GROUP BY` across the transactions table.

In the document model, each transaction is an isolated document. Cross-document aggregation is possible (e.g., MongoDB's aggregation pipeline) but is not the model's architectural strength — it's optimised for single-document reads, not cross-entity analytics.

Now imagine two transactions arrive simultaneously — one crediting \$500, one debiting \$300. Both read the account's balance document:

Predict: What happens to the ledger?

NeoBank: The Lost Update

Time	Txn A (credit \$500)	Txn B (debit \$300)
t_1	Read balance $\rightarrow$ \$1,000	
t_2		Read balance $\rightarrow$ \$1,000
t_3	Write: $\$1,000 + \$500 = \$1,500$	
t_4		Write: $\$1,000 - \$300 = \$700$ X
	Final balance: \$700	Expected: \$1,200

Both write their result. **One update is silently lost.** The ledger is now wrong, and no constraint prevented it.

NeoBank: Guarantee Mismatch

Some document databases offer opt-in multi-document transactions — but bolting cross-document guarantees onto a model designed for isolated reads **fight the grain**. Coordination overhead erodes the performance advantage; transaction boundaries must be managed in application code.

The document model optimises for *self-contained reads*. This workload requires *cross-document aggregation* and *transactional consistency*. This is a **guarantee mismatch**.

The mismatch isn't about shape — the data looked fine structurally. It's about what the model **is built to guarantee**.

The Pattern

Neither failure is a bug. Both systems **functioned**.

The failures are **structural**: the storage model did not match the workload.

	RideLink	NeoBank
Model used	Relational	Document
Problem	Correct but slow & brittle	Accessible but unprotected
Root cause	Shape mismatch	Guarantee mismatch
Resolution	Migrated to MongoDB	Moved to PostgreSQL

Both teams paid: **months of re-engineering, migration risk, and downtime**.

Therefore, the engineer must **match the storage model to the workload before committing** — not discover the mismatch in production.

What Must an Engineer Know?

What capabilities would have prevented these failures?

1. **Identify** workload characteristics — otherwise you default to the familiar
2. **Match** model to workload — otherwise shape or guarantee mismatch
3. **Articulate** trade-offs — otherwise the choice is accidental
4. **Recognise** mismatch from symptoms — otherwise you tune when you should migrate
5. **Use precise vocabulary** — otherwise imprecise reasoning → wrong decisions

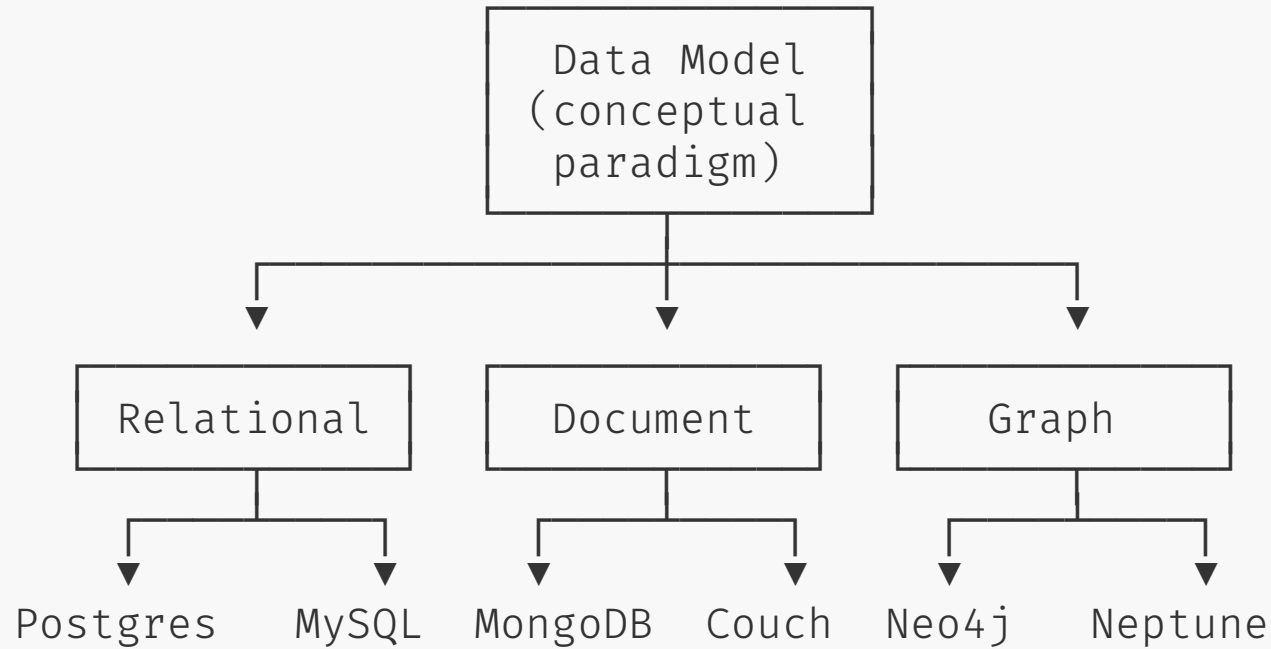
Capability 5 demands precise language — so we begin with **vocabulary**.

Course Vocabulary: Five Terms

Term	Definition	Example
Data model	Structural paradigm for organising and querying data	Relational, document, property graph
Workload	Dominant access pattern of a system	Transactional, personalisation, traversal
Schema	Structural contract — types, constraints, relationships	CREATE TABLE, XSD, ontology
Constraint	Rule enforced by the schema	order_id is unique
Query	Declarative request for data	"Running balance of A-1001"

Predict: which of these five terms will matter most for *distinguishing* one model from another?

Data Model ≠ Database

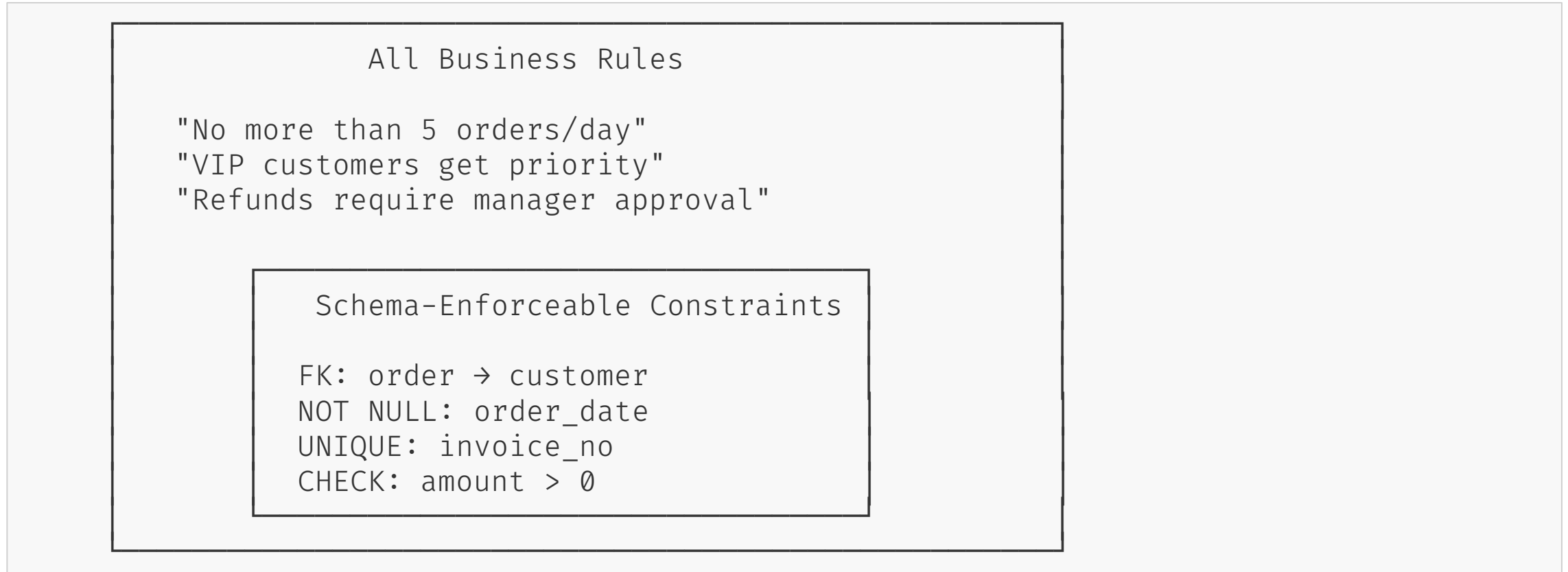


- **Data model** = conceptual paradigm (a way of structuring and connecting data)
- **Database** = running system that implements a model (PostgreSQL, MongoDB)

Choose the model first, then the product.

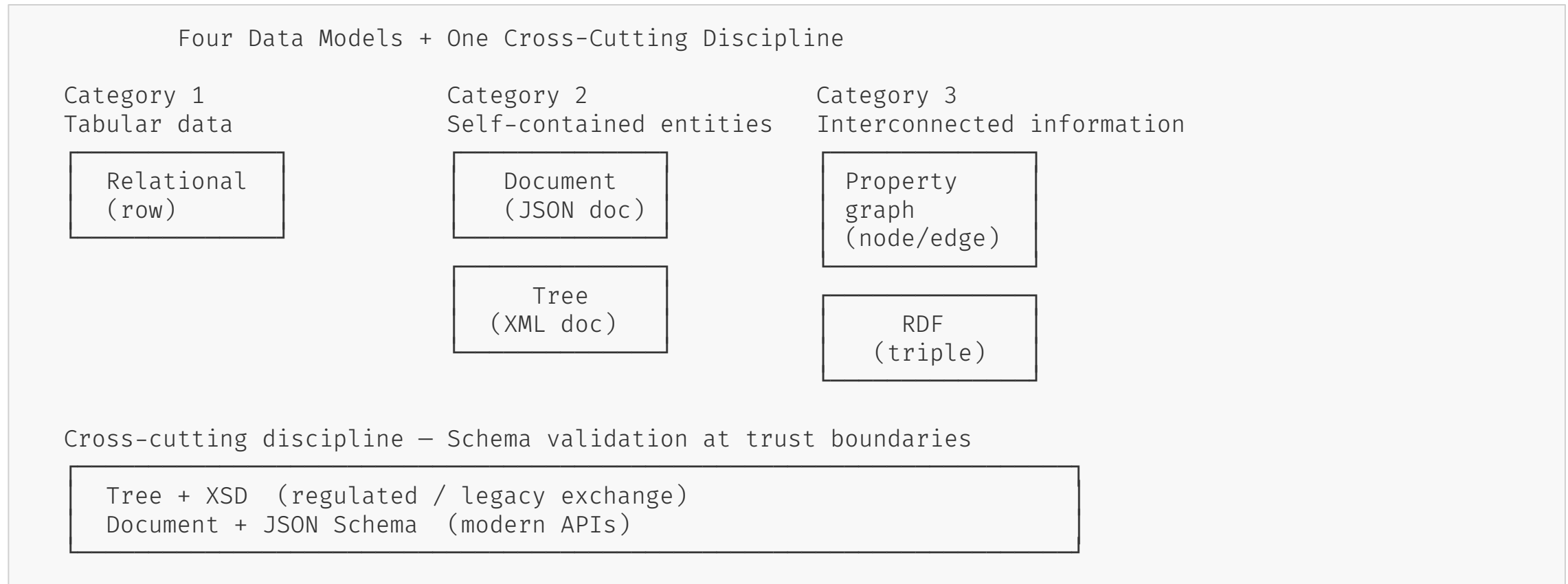
Constraint ≠ Business Rule

A **constraint** is the subset of business rules that the *schema can enforce*.



If you assume the schema handles a rule it cannot enforce, you discover the gap in production — like NeoBank's missing cross-document consistency.

Four Models: The Big Picture



Three categories + one cross-cutting discipline. Each category reflects a different structural assumption about how data relates.

Four Models: Summary

Category	Models	Structure
1 — Tabular	Relational	Flat rows, enforced schemas
2 — Self-contained	Document, XML Tree	Nested entities, flexible schemas
3 — Interconnected	Property Graph, RDF	Nodes/triples, relationships first
Cross-cutting	Schema Validation (XSD, JSON Schema)	Contract enforcement at boundaries

We'll introduce each model through examples — starting with one you've already seen.

Relational Model

Unit of storage	Row in a table
Connection mechanism	Foreign keys + JOIN
Schema enforcement	DDL (Data Definition Language) — strict, schema-on-write

The engine **rejects** invalid data on every write: primary keys, foreign keys, NOT NULL, CHECK, UNIQUE.

This is the first model's "property card." Watch how the next three models differ on exactly these three dimensions.

Relational: RideLink's Tables

Three tables — flat rows, spread across tables, connected by `user_id`:

Users

user_id	name	phone
P-3921	Mei Lin	+65-9123-4567

Addresses

addr_id	user_id	label	street
1	P-3921	Home	12 Orchard Rd
2	P-3921	Office	88 Robinson Rd

PaymentMethods

pm_id	user_id	type	last4	is_default
1	P-3921	visa	4242	true

To load the full profile, the application must **JOIN all three tables** on `user_id`. The result is flat rows — the application must re-nest them into the JSON the mobile app expects.

Document: RideLink as JSON

Now the same passenger profile — **everything nested in one place:**

```
{
  "user_id": "P-3921",
  "name": "Mei Lin",
  "phone": "+65-9123-4567",
  "addresses": [
    { "label": "Home", "street": "12 Orchard Rd" },
    { "label": "Office", "street": "88 Robinson Rd" }
  ],
  "payment_methods": [
    { "type": "visa", "last4": "4242", "default": true }
  ]
}
```

One read returns the entire profile. No JOINS. No reconstruction.

Document Model

Unit of storage	JSON/BSON (Binary JSON) document
Connection mechanism	Embedded sub-documents; references by ID (application-managed)
Schema enforcement	Optional — schema-on-read by default; opt-in validation available

The document is self-contained and nested. Relationships *within* a document are expressed through nesting. Relationships *across* documents are application-managed — no referential integrity equivalent to relational foreign keys. Delete a user document — no cascade deletes their orders. That's your code.

Same Data: What Changed?

	Relational	Document
Storage	3+ normalised tables	1 nested document
Read	Multi-table JOIN	Single key lookup
Write (new section)	New table + migration	Add field to document
Cross-entity query	Native (GROUP BY)	Expensive (\$lookup)
Referential integrity	FK enforced	Application-managed

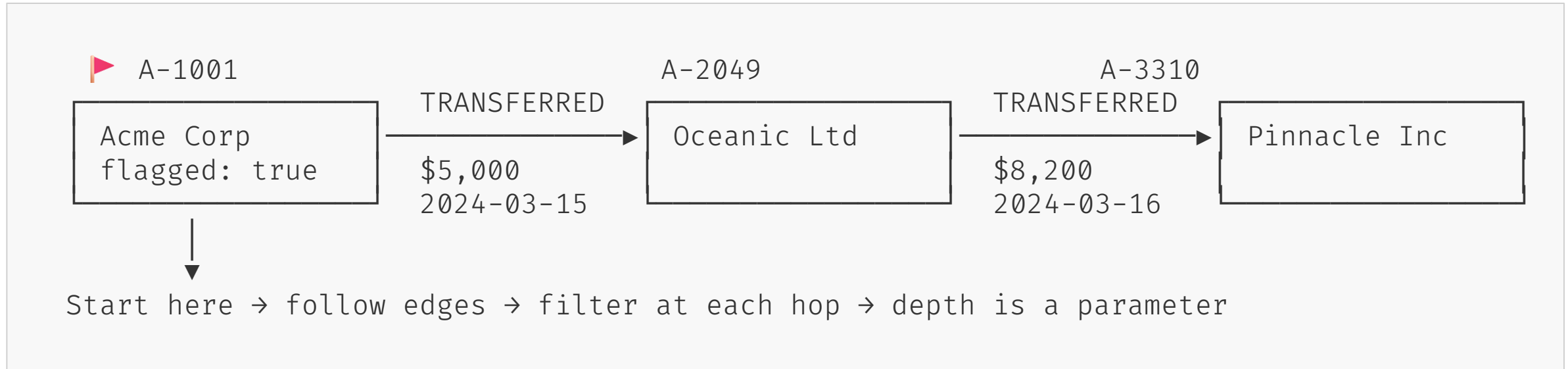
Top three rows: shape alignment benefits. Bottom two rows: the trade-off costs RideLink accepted.

The storage shape now matches the delivery shape. That's **shape alignment** — and it explains why RideLink eventually migrated to MongoDB.

Predict: NeoBank's fraud team traces money flows by hopping between accounts. Can either model handle this efficiently?

NeoBank Again: The Fraud Team

Same company, different workload. NeoBank's fraud team needs to trace money flows from a flagged account.



The analyst starts at the flagged node → traverses outward → filters by amount/date at each hop. **No JOINS.**

What Makes This Traversal Work?

This is a **property graph** — nodes and edges, both carrying typed properties.

Unit of storage	Node with properties
Connection mechanism	Labelled, directed edges with properties
Schema enforcement	Labels + application logic — schema-optional

Relationships are **first-class citizens**, not afterthoughts reconstructed by JOINS. The model is optimised for *traversal* — following chains of relationships with filtering at each hop.

Property Graph: Traversal, Not JOINS

The analyst's query: "start at A-1001, follow transfer edges up to 3 hops, filter by amount."

In the property graph, this is a **single traversal** — start at a node, follow edges, apply filters at each hop.

No JOINS. No recursion. Changing from 3 hops to 5 is a **parameter change**, not a structural rewrite. In relational, each hop adds another self-join — and the result set can grow **exponentially**.

	Relational	Property graph
3 hops	3 self-joins (query rewrite)	depth: 3 (parameter)
5 hops	5 self-joins (query rewrite)	depth: 5 (parameter)
N hops	N joins; query grows structurally	Parameter change only

NeoBank Again: The Compliance Team

Same company, third workload. NeoBank compliance must merge screening data from **3 independent sources** for customer due diligence.

Names are ambiguous — "Ahmad Ibrahim" could be two different people. **URIs (Uniform Resource Identifiers) provide globally unique identity.**

```
cust:C-4821  cust:riskTier      "standard" .  
cust:C-4821  pep:pepStatus     pep:ActivePEP .  
cust:C-4821  sanc:matchStatus  sanc:NoMatch .
```

All three use the **same URI** (`cust:C-4821`). A query can combine facts from all three — without a pre-agreed schema or ETL pipeline. The URI is the join key.

What Makes This Integration Work?

This is **RDF** — every fact is a triple (subject–predicate–object), every entity has a globally unique URI.

Unit of storage	Triple (subject–predicate–object)
Connection mechanism	Predicates connect resources via URIs
Schema enforcement	Ontology (RDFS/OWL) — descriptive, not enforced

Nobody controls all three sources. The URI **is** the integration mechanism — no pre-agreed schema, no ETL (Extract, Transform, Load). If the screening provider adds adverse media results later, those triples attach to the same URI automatically.

Schema Validation: The Cross-Cutting Discipline

GlobalPay needs to send a credit-check request to a bank. Neither company controls the other's systems. How do they agree on what a valid request looks like?

Not a storage model — operates **across shapes** at **trust boundaries**. (XML tree appeared in Category 2 — structurally it's a self-contained hierarchy like a JSON document — but its dominant real-world use is here, paired with XSD (XML Schema Definition) at trust boundaries.) When data crosses an organisational boundary, the **schema is the only shared artefact** and **validation is the only defence**.

Schema Validation: Two Formats, One Discipline

Format	Schema language	Primary context
XML	XSD, DTD	Regulated exchange — e-invoicing, payment messages
JSON	JSON Schema	Modern APIs — REST contracts, webhooks

The discipline is the same regardless of format:

1. **Read** the contract (schema)
2. **Validate** before submission
3. **Extract** after validation

Schema Validation: Credit-Check Request

A credit-check request (GlobalPay → bank) as XML:

```
<CreditCheckRequest id="CR-20240315-001">
  <Applicant>
    <FullName>Mei Lin Tan</FullName>
    <NationalID>S9012345A</NationalID>
    <DateOfBirth>1990-03-15</DateOfBirth>
  </Applicant>
  <RequestedAmount currency="SGD">50000.00</RequestedAmount>
  <Purpose>Home renovation</Purpose>
</CreditCheckRequest>
```

The bank publishes an **XSD** — the contract that defines what valid requests look like.

A request missing `DateOfBirth` or with a malformed `id` → **rejected before it reaches any system.**

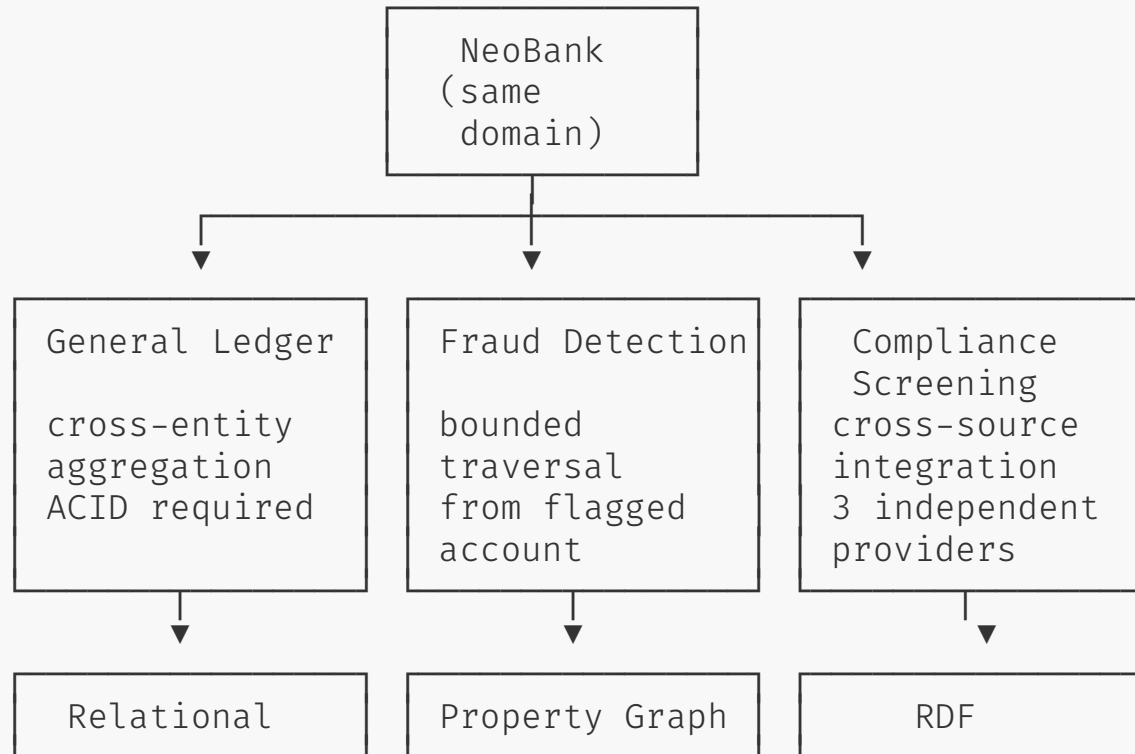
Schema Validation: The XSD Contract

The bank's published XSD (excerpt):

```
<xs:element name="CreditCheckRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Applicant"> !— FullName, NationalID, DateOfBirth — </xs:element>
      <xs:element name="RequestedAmount">
        <xs:complexType>
          <xs:simpleContent>
            <xs:extension base="xs:decimal">
              <xs:attribute name="currency" type="xs:string" use="required"/>
            </xs:extension>
          </xs:simpleContent>
        </xs:complexType>
      </xs:element>
      <xs:element name="Purpose" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:string" use="required"/>
  </xs:complexType>
</xs:element>
```

DateOfBirth has type="xs:date" — submit "not-a-date" and the validator rejects it.
RequestedAmount carries a required currency attribute — omit it and the validator rejects the request.

NeoBank: Three Workloads, Three Models



The domain is the same; the **workloads** are different.

Model selection follows **workload**, not domain.

Callback: Which Term Distinguishes Models?

Earlier we asked: which vocabulary term most sharply distinguishes one model from another?

Schema enforcement.

Every model card included it — and every model answered differently:

- Relational → engine enforces on write
- Document → schema-on-read by default
- Property graph → schema-optional
- RDF → ontology is descriptive, not enforced
- XSD/JSON Schema → validator enforces at the boundary

The **enforcement spectrum** is the organising principle of this topic.

What Happens When You Insert Bad Data?

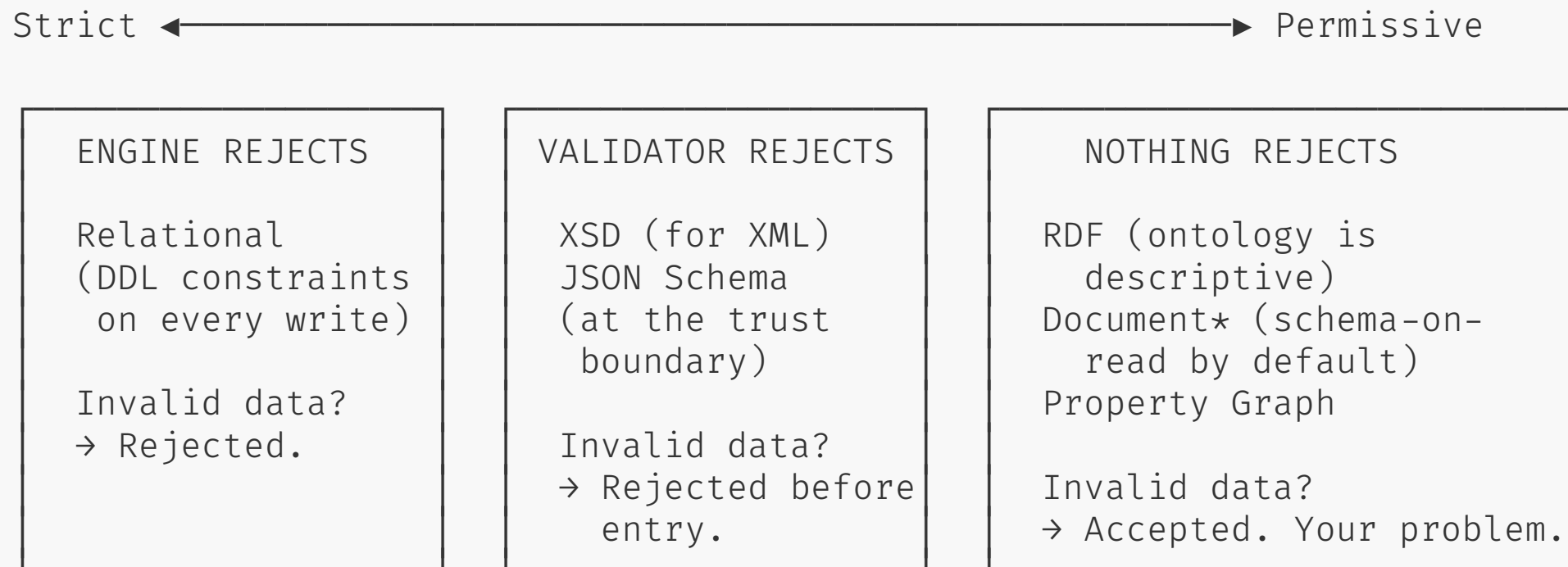
Imagine two inserts:

1. **MongoDB** (no validation rules): insert a document where `price` is set to `"abc"` (a string).
2. **PostgreSQL**: insert a row where a `price NUMERIC NOT NULL` column receives `"abc"`.

Predict: What does each system do — accept or reject?

The Enforcement Spectrum

The document database accepts the insert. PostgreSQL rejects it with a type error. Where a model sits on this spectrum tells you what you get for free:



* Document is schema-on-read by default; opt-in validation available.

Five Recurring Concepts

What properties must every data model answer for? These five appear in **every** model — but each supports them differently.

Concept	Definition
Schema (S)	Declared structure — types, constraints, relationships
Instance (I)	Actual data conforming to a schema (if schema = blueprint, instance = the building)
Constraint (C)	Invariant the system enforces
Query (Q)	Declarative request; returns derived data
Transaction (T)	Atomic unit of work with guarantees

Not all models support all five equally — these **asymmetries** are the source of every trade-off.

What Each Model Optimises For

C (constraints) and **T** (transactions) — weakened at the cross-document level. That's exactly what broke NeoBank's ledger.

Model	Optimises for	Costs	Workload
Relational	ACID + cross-entity queries	Schema rigidity; write scalability (cf. RideLink)	Banking, orders, inventory
Document	Aggregate reads + flexibility	Cross-doc queries expensive (cf. NeoBank auditor)	Profiles, CMS, catalogues

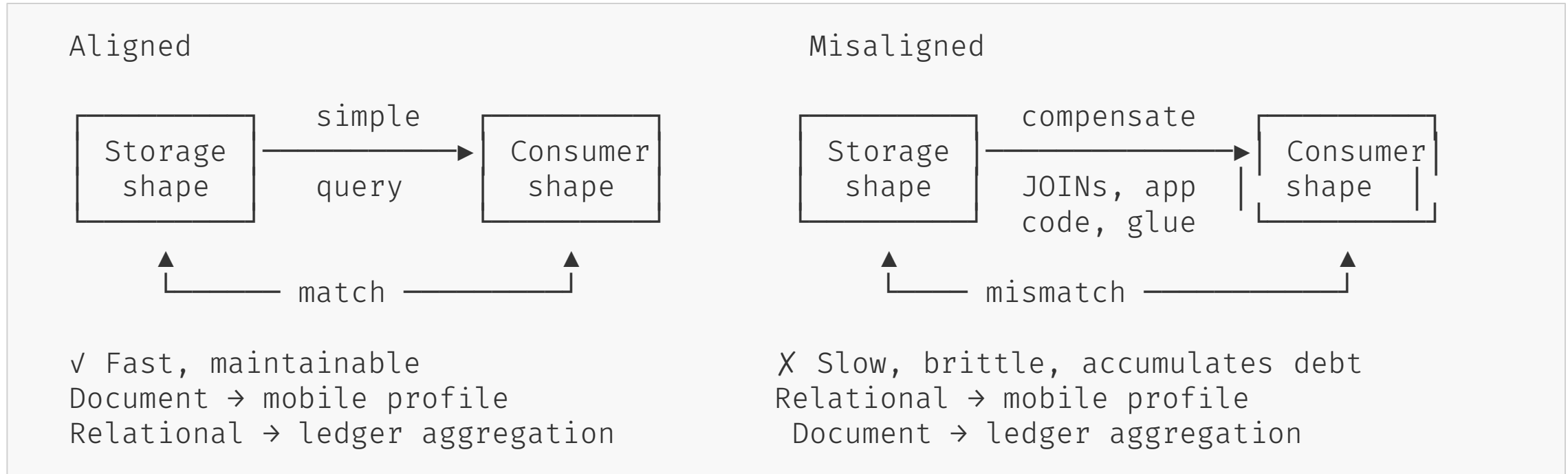
The remaining three models follow the same trade-off pattern.

What Each Model Optimises For (continued)

Model	Optimises for	Costs	Workload
Property graph	Relationship traversal	Global aggregation expensive	Fraud, social, recommendations
Schema validation	Contract enforcement	Verbosity; maintenance cost	e-invoicing, REST APIs
RDF	Cross-source integration	Storage overhead; steep learning curve	Compliance, biomedical, libraries

No model is universally superior. Each trades one guarantee for another.

Shape Alignment: The Core Principle



Shape alignment is the guiding principle — not an absolute rule. (CQRS (Command Query Responsibility Segregation) deliberately misaligns shapes, accepting synchronisation cost for independent scalability.) But given a new system, how do you decide which model's shape aligns? We need a procedure.

Why Not Just Relational? (1/3)

The relational model is the **most general-purpose** model. If you learn only one, learn relational.

Task A — Load the full profile for passenger P-3921.

Delivery shape: a nested document (personal details + arrays of addresses, payments, preferences).

In relational: data lives in 5 tables → 4 JOINS → flat result set → re-nest in application code.

In document: storage **is** the delivery shape — one read, one document, done.

The relational model can do it — but shape misalignment makes it expensive.

Why Not Just Relational? (2/3)

Task B — Starting from A-2049, find all accounts reachable within 3 transfer hops where each transfer exceeds \$5,000.

	Relational	Property graph
3 hops	3 self-joins (query rewrite)	depth: 3 (parameter)
5 hops	5 self-joins (query rewrite)	depth: 5 (parameter)
N hops	N joins; query grows structurally	Parameter change only

The relational model can do it — but the query fights the storage shape. Traversal is what graphs optimise for.

Why Not Just Relational? (3/3)

Task C — Merge screening results from 3 independent databases with different schemas and identifiers.

In RDF: each source publishes triples with a shared URI → query combines facts → no ETL.

In relational: design a unified compliance schema + write 3 ETL pipelines + maintain identifier mappings. Nobody controls all three schemas. Adding a new source means another ETL pipeline; in RDF, new triples attach to existing URIs automatically.

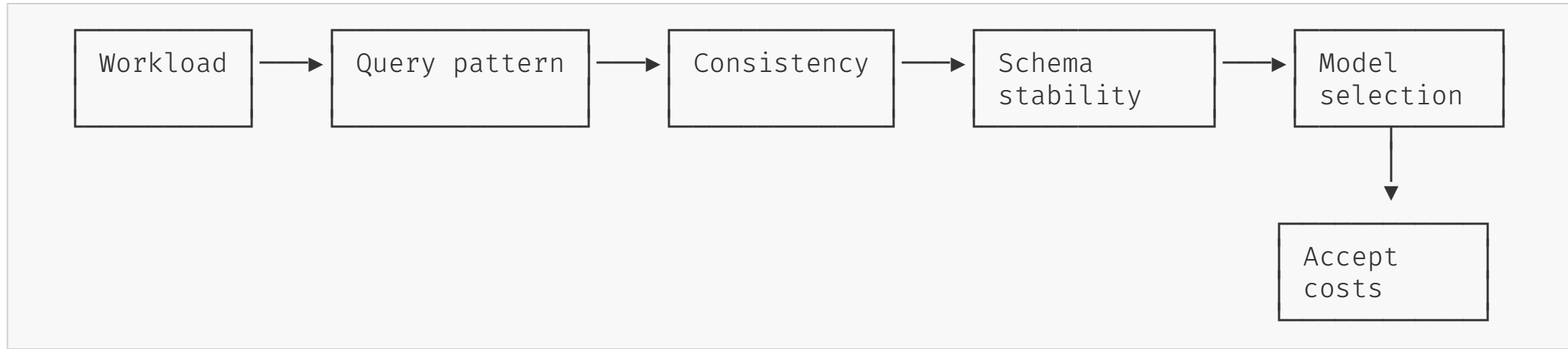
Why Not Just Relational?: The Pattern

Task	Access pattern	Fights relational because...	Natural in
A Profile load	Aggregate read	Storage $\neq$ delivery shape; 5-table JOIN	Document
B Transfer hops	Bounded traversal	N self-joins; depth hard-coded	Property graph
C Cross-source merge	Integration	One-authority schema assumption	RDF

Three tasks, three **fundamentally different** structural mismatches — not one recurring problem.

None of these tasks is *impossible* in relational — just **awkward**. More code, more workarounds, more maintenance.

The Decision Framework



Given a data storage problem, apply four steps:

1. **Identify** the primary query pattern
2. **Assess** consistency requirements
3. **Evaluate** schema stability
4. **Match** to model — and **explicitly state the costs** you are accepting

Steps 1–3 are analytical. Step 4 is the discipline: **name the trade-off out loud.**

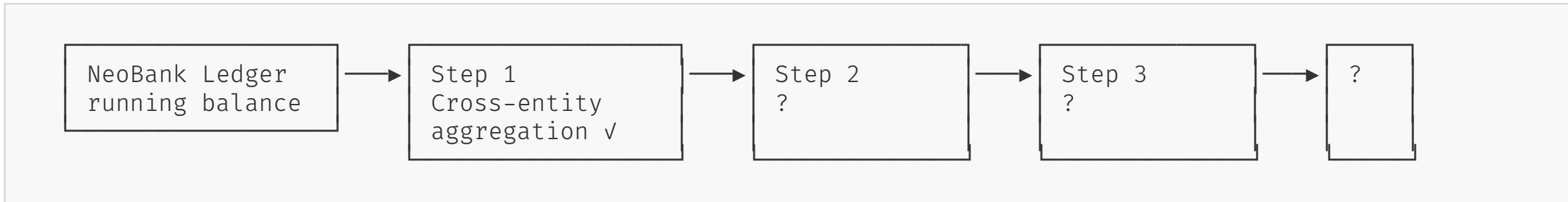
Step 1: What Does the System Mostly Do?

Pattern	Description	Points toward
Aggregate read	Load one entity with all related data	Document
Cross-entity aggregation	SUM, COUNT, GROUP BY across entities	Relational
Bounded traversal	Follow relationships N hops with filters	Property graph
Schema-validated exchange	Structured data validated against a contract	Schema validation
Cross-source integration	Merge data from independent sources	RDF

NeoBank's ledger? Cross-entity aggregation → relational. RideLink's profile? Aggregate read → document.

Step 1: Quick Trace: NeoBank's Ledger

Return to NeoBank's ledger. Dominant operation: "compute the running balance of A-1001 across all transactions."



Step 1 points toward **relational**. Steps 2 and 3 will either confirm or challenge this.

Step 2: What Consistency Does It Need?

Requirement	Description	Points toward
ACID transactions	Multi-step operations must be atomic	Relational
Eventual consistency	Temporary inconsistency acceptable	Document
Open-world assumption	Absence $\neq$ false, just unknown	RDF
Application-managed	Consistency in application code	Document, Property graph

This is the step that broke NeoBank. The document model was fine for reads — but the workload *demand*ed ACID, and the model isn't built to provide it. Even with opt-in transactions, you're fighting the grain.

Step 3: How Stable Is the Schema?

Stability	Description	Points toward
Fixed schema	Structure known, stable, rarely changes	Relational; XSD/JSON Schema
Evolving schema	Structure changes with business needs	Document
Schema-late	Structure emerges from data	RDF
Schema-optional	Labels and conventions, loose structure	Property graph

RideLink's profile sections evolved as the product grew — evolving schema → document. NeoBank's ledger is fixed, tabular, regulated — fixed schema → relational. This step often confirms what Step 1 suggested, but when it disagrees, that's a signal to look more carefully — the workload may need to be split.

Step 4: Match to Model; Accept Costs

1. Select the model whose **strength** aligns with your query pattern, consistency, and schema stability
2. Consult the **trade-off table** — explicitly state the costs you are accepting
3. If you **cannot accept the costs** → revisit assumptions; the workload may need to be **split across models** (Topic 11)

Two engineers may weigh factors differently and arrive at different models. That's fine — the discipline is making the reasoning **explicit**, not finding a universal rule.

The framework produces a **justified choice**, not a universal answer.

FreshPlate: A Recipe-Sharing Platform

FreshPlate: home cooks publish and browse recipes.

Primary operation: "Load the complete recipe for display" — title, ingredients, steps, photos, ratings, comments.

Different recipe types have **different structures** (baking vs cocktail vs meal-prep).

New sections added **every quarter** (nutritional info, video links, allergens).

Predict: which model does the framework select? Which step is most decisive?

FreshPlate: Applying the Framework

Step	Assessment	Justification
Query pattern	Aggregate read	One API call loads full recipe with all related data
Consistency	Eventual	No multi-recipe transactions; brief staleness OK
Schema stability	Evolving	Different types, new sections quarterly
Model	Document	All three steps align
Costs accepted	Cross-recipe analytics require scanning all docs; no referential integrity for separate author profiles	

FreshPlate: What About the Costs?

Marketing asks: "What is the average rating across all Italian recipes published this month?"

This is **cross-document aggregation** — the document model handles it less efficiently than relational's native `GROUP BY`.

Options: MongoDB aggregation pipeline (scans matching docs) or a separate analytics store.

If this analytical workload grows to dominate, the team may need a secondary relational store for reporting — a **multi-model design** (Topic 11).

When Steps Disagree

FreshPlate was clean — all three steps pointed to **document**.

Real systems are messier:

- Step 1 → document (aggregate reads)
- Step 2 → relational (ACID required)

When steps conflict:

- Weigh which factor **dominates**
- Accept the costs of the chosen model for the factors it doesn't serve
- If costs are intolerable → **split the workload** across models (Topic 11)

Check 1: Hospital Patient Records

A hospital's primary operation: "Load the complete patient record for the doctor's screen" — demographics, diagnoses, prescriptions, lab results, visit history.

Patient record structures vary (paediatric vs oncology vs cardiology) and evolve as clinical standards change.

Which model? Which step is most decisive?

Step 1: aggregate read. Step 3: evolving schema. Both point to **document**. Cost: cross-patient analytics ("average length of stay by diagnosis") require scanning all documents — a reporting workload relational handles natively.

Check 2: Government Tax Returns

A government tax authority receives annual tax returns from employers. Each return must conform to a **legally mandated schema** (published as an XSD). Returns that fail validation are rejected with error codes. The authority processes **2 million returns** per filing season.

Which model? Why can't relational **CHECK** constraints handle this?

Step 1 → **schema-validated exchange** — data crosses an organisational boundary. Validation must happen *before* data enters any system. → **Schema validation (XML/XSD)**. Relational **CHECK** constraints operate at insert time — they cannot reject a document before it reaches the database. Cost: XSD schemas are verbose and maintenance-heavy.

Check 3: E-Commerce Platform

Orders, inventory, payments. When a customer places an order: inventory decremented, payment created, shipping label generated — **atomically**. If any step fails, all roll back.

The CFO needs: total revenue by category, average order value, top customers.

What's the decisive workload characteristic?

Step 2: ACID transactions are mandatory — partial orders corrupt inventory counts. Schema is fixed and stable. → **Relational**. Cost: schema rigidity (adding "gift wrapping" means `ALTER TABLE`); high write volumes may also require sharding.

Checks 1–3: The Pattern So Far

Check	Decisive step	Model
Hospital	Step 1 — aggregate read	Document
Tax returns	Step 1 — schema-validated exchange	Schema validation (XSD)
E-commerce	Step 2 — ACID overrides Step 1	Relational

Three checks — two resolved at Step 1 (different query patterns), one at Step 2 (consistency override). Three different models. No single step always dominates — the workload determines which step is decisive.

The remaining checks test **harder cases** — failure diagnosis, cross-source integration, and conflicting signals.

Check 4: Music Streaming Playlists

A music streaming service stores playlists in a relational database: `Users`, `Playlists`, `PlaylistSongs`, `Songs`. Each playlist display requires a 3-table JOIN. Users report the "My Library" page takes **4 seconds** to load.

The team suspects an indexing problem. Is it?

Shape mismatch, not indexing. The delivery shape (nested list of playlists with songs) does not match the storage shape (flat, normalised tables). The document model would store each playlist as a self-contained document — one read, no JOINs, sub-100ms. The team is treating a **model-workload mismatch as a tuning problem**. Cost: cross-playlist analytics ("most-added songs this week") require scanning all documents.

Check 5: Pharma Clinical Trials

A pharmaceutical company runs clinical trials across **12 hospitals**. Each hospital records data in its own system with its own schema.

A researcher asks: "Which patients across all sites received Drug X and showed improvement in Biomarker Y?"

What makes this structurally different from the previous checks?

The data *already exists* in multiple independent sources with **no shared schema** — this is cross-source integration. → **RDF**. Each hospital publishes triples using shared URIs. Cost: simple lookups ("find patient P-1234") are verbose in SPARQL; steep tooling learning curve. Relational alternative: master schema + 12 ETL pipelines — grows linearly with each new site.

Checks 4–5: Harder Cases

Check	What it tests	Lesson
Music streaming	Failure diagnosis	Shape mismatch $\neq$ tuning problem — diagnose before you optimise
Pharma trials	Cross-source integration	RDF when no one controls the schema

Both checks required reasoning **before** the framework: recognising that the team misidentified the problem (Check 4) and recognising that the data landscape itself dictates the model (Check 5).

The final check tests the hardest case — what happens when framework steps **disagree**.

Check 6: Ride-Sharing (Conflicting Signals)

A ride-sharing company tracks trips for driver payouts.

- **Finance** needs monthly aggregates (total payouts per driver, average fare by zone) with **ACID guarantees**
- **Product** adds new trip fields every quarter and wants the complete trip loaded in **one call**

Step 1 points to two different models. Which two, and which step breaks the tie?

Finance → **relational** (cross-entity aggregation). Product → **document** (aggregate read, evolving schema). Step 2 breaks the tie: **ACID is mandatory** → relational. Cost: quarterly `ALTER TABLE` migrations. If the cost becomes intolerable → split across models (Topic 11).

The Pattern Across All Six

Each check tested a different aspect of the framework:

Check	Decisive factor	Model	What it tests
Hospital	Step 1 — aggregate read	Document	Straightforward application
Tax returns	Step 1 — schema-validated exchange	Schema validation (XSD)	Cross-boundary enforcement
E-commerce	Step 2 — ACID overrides Step 1	Relational	Consistency dominates
Music streaming	Shape mismatch diagnosis	Document	Failure ≠ tuning problem
Pharma trials	Cross-source integration	RDF	No one controls the schema
Ride-sharing	Steps 1 & 2 conflict	Relational	Conflicting signals

No single step always dominates. The **workload** determines which step is decisive.

Revisiting the Failures

Return to the opening. RideLink stored nested profiles in flat relational tables. NeoBank stored a financial ledger in isolated documents.

Now we can diagnose both precisely:

	RideLink	NeoBank
Mismatch type	Shape mismatch	Guarantee mismatch
Framework step	Step 1 — aggregate read → document	Step 2 — ACID required → relational
Resolution	Migrated to MongoDB	Moved to PostgreSQL

Two failures, two diagnoses, two different steps in the same framework. That's the discipline — **explicit reasoning about workload and model**, not instinct.

Course Map

Topic 1 (this topic – vocabulary, decision framework, four models + validation surveyed)

— Topics 2–5: Relational deep-dive

— Topic 2: ERD – capturing business rules before committing to tables

— Topic 3: Normalisation – eliminating redundancy; BCNF vs star schema

— Topic 4: SQL – querying with JOINS, aggregation, CTEs

— Topic 5: Enforcement – constraints, transactions, indexes, access control

— Topics 6–10: Alternative models + cross-cutting discipline

— Topic 6: Document (MongoDB) – revisits Failure 1 (RideLink profiles)

— Topic 7: Schema validation – XSD and JSON Schema at trust boundaries

— Topic 8: RDF – knowledge graphs; global identifiers; open-world assumption

— Topic 9: SPARQL – querying knowledge graphs (builds on Topic 8)

— Topic 10: Property graph (Neo4j/Cypher) – bounded traversal

— Topic 11: Capstone – multi-model design (all four models + validation, one system)

Concepts That Recur Throughout the Course

Five ideas planted today — each appears briefly here and grows into a full mechanism in later topics.

Concept	Introduced here as	Fully developed in
Enforcement spectrum	"engine rejects / validator rejects / nothing rejects"	Topics 5, 7, 8
Shape alignment	"storage shape vs delivery shape"	Topics 3, 6, 11
Schema as contract	"the schema is a contract between parties"	Topic 7
Workload not domain	"the workload may need to be split across models"	Topic 11
Trade-off discipline	"explicitly state the costs you are accepting"	Every topic

Next: Topic 2 Conceptual Modelling (ERD)

The relational model won two of the six scenarios above.

But writing `CREATE TABLE` directly from requirements is risky — **business rules get lost in translation.**

Topic 2 introduces **Entity-Relationship Diagrams** as a design tool that captures business rules *before* committing to tables.

Lab Preview

Reasoning lab — no code, no database installation.

- **Part A** — Concept check: 6 MCQ questions with justifications
- **Part B** — Scenario analysis: apply the decision framework to 3 extended scenarios
- **Part C** — Synthesis: connect across scenarios

All answers must use the **formal vocabulary** and reference **specific tables by number**.

Lab file: `01-lab-scenario-reasoning.md`